

Nicole González

Informe Ejecutivo

Diplomados Online antes carecía de un sistema automatizado para el registro de estudiantes y de profesores con un Excel que en ocasiones daba problemas con datos repetidos, ante esto me asegure de que el código arroje error en caso de que un dato se repita, con el sistema realizando una búsqueda en la memoria para evitar confusiones en registros y notas asegurando de esta manera las consistencias en futuros reportes.

En el caso de las notas los sistemas en los tres lenguajes el sistema acepta tres notas separadas por comas, almacenando los datos de forma síncrona en archivos txt garantizando la persistencia de la información ante apagones repentinos.

Otro fallo que se encontró fue en los cálculos de los promedios debido a que cada programa educativo tiene sus normas de aprobación, es decir, las normas de aprobación no son las mismas en los diplomados y en los cursos, esto tomaría mucho tiempo en sacarlo en un archivo Excel y para el caso se tendrían que hacer tres archivos una para el curso, otro para el Bootcamp y otro para los diplomados, uno de los requerimientos exigía que en el Bootcamp no se promediara, sino que se validara el hecho de que ninguna nota individual fuera menor a 14. El calculo del promedio final presentaba desviaciones severas cuando entre los registros de los estudiantes se encontraban incompletos, durante el proceso de creación de los códigos se incorporó una regla matemática estricta que penaliza las evaluaciones no presentadas asignándoles un valor de 0, dividiendo de manera invariable el puntaje acumulado entre la totalidad de las evaluaciones programadas (en este caso serían tres), de manera que, una calificación única de 18 se procesa como $(18+0.0+0.0)/3$, arrojava una calificación definitiva de 06. Este criterio unificado se acopla con el requerimiento del formato para el Bootcamp, el cual

rompe el esquema promedio ponderado y exige una evaluación individualizada para certificar que ninguna nota sea menor a 14.

Uno de los requerimientos también fue aplicar el polimorfismo, lo cual se logró, cada alumno arrastrara una instancia directa de su modalidad académica. Cuando el sistema necesita saber si alguien aprobó, invoca una función virtual común y el objeto responde con su propia regla interna para cada programa. Transformando una infraestructura rígida en un entorno escalable capaz de recibir nuevas ofertas formativas sin que ocurra una alteración en la lógica central.

Programar en tres lenguajes:

Programar primero en Python fue un proceso más fluido y sencillo, debido a que al ser un lenguaje interpretado con un tipado dinámico y una gestión automática de memoria, permitió el enfoque en la lógica de las estructuras de datos, además de que la flexibilidad de sus listas facilitó el empaquetado en las notas mediante funciones simples como `join()` y rebanados de texto.

En Java se pudo ver que el lenguaje es más rígido debido a los tipos estrictos ante la necesidad de usar colecciones oficiales como `Stack` y `LinkedList`. Sin embargo, el compilador funcionó como guía de forma clara, el manejo de excepciones con bloques `try-catch` y las expresiones funcionales ayudó para poder blindar el menú del programa.

En C++ el entorno compilado obligó a lidiar con la gestión manual de memoria a través de punteros como `new` y `delete`, en donde además un solo error en la colocación de una llave rompía con el alcance que tenían las variables, desatando en una avalancha de errores crípticos en la terminal como `extra qualification` o declaraciones fuera de scope. Este lenguaje exigió

balancear cada estructura a detalle y lidiar con las restricciones de seguridad en la consola de Windows, que bloqueaba la ejecución de los binarios locales.

Cuadro Comparativo

Lenguajes	Ventajas	Desventajas
Python	• Programación funcional con comprensiones de listas para flujos declarativos. • Al no compilar, los cambios en la lógica LIFO/FIFO se probaron al instante. • Las listas nativas mutan de tamaño solas, facilitando el procesamiento al picar las notas por comas. • No hay que preocuparse por tipos de datos ni llaves rotas.	• Los fallos de tipeo o lógica se descubrían solo cuando el usuario ejecutaba esa línea en la terminal. • Las variables no son privadas de verdad (cualquier método externo puede alterar las notas por accidente).
Java	• El compilador frena errores de asignación antes de ejecutar, obligando a mapear bien las clases Alumno y Profesor. • El uso de Stack para el botón de pánico y Queue para los certificados fue directo y seguro. • Encapsulamiento real con modificadores de acceso (private/protected) infalibles, bloqueando de forma estricta cualquier intento de alterar las notas o datos del alumno desde fuera de la jerarquía autorizada. Esto obliga al sistema a usar métodos controladores (getters y setters), garantizando así la integridad absoluta de la información en la memoria RAM y previniendo la corrupción de los archivos de texto.	• Exige declarar clases e interfaces para todo, ralentizando la velocidad de codificación inicial. • Configurar los flujos de lectura de archivos requiere capturar múltiples excepciones obligatorias de entrada/salida.
	• Velocidad nativa de ejecución. Permitiendo decidir el milisegundo exacto en que un objeto se borra de la RAM. • Polimorfismo impecable: El uso de funciones virtuales fuerza a la CPU a resolver la regla	• Una llave de cierre mal puesta (}) partía la clase SGA a la mitad, dejando las funciones globales aisladas de las variables miembro (listaAlumnos).

C++	académica exacta en tiempo de ejecución, eliminando los if por completo.	•Olvidar el operador de resolución de ámbito (SGA::) al declarar métodos fuera de la clase o dejarlo duplicado dentro causaba fallos de extra qualification. •Restricciones severas en PowerShell al procesar cadenas con rutas físicas completas (Token inesperado). •Olvidar un delete provocaba que la memoria del heap se quedara secuestrada.
------------	--	--

La implementación de estos sistemas demuestra que el éxito de un software no depende de una programación estructurada tradicional, sino de una correcta elección de la arquitectura. El usar el polimorfismo real redujo el acoplamiento del código logrando que el SGA pueda escalar y recibir nuevas modalidades de estudio en el futuro sin modificar el programa principal.

El desarrollo en tres lenguajes diferentes dejó una lección en cuanto a la gestión de recursos, debido a que lenguajes con recolectores de basura como en Java y Python se priorizan la productividad, C++ prioriza la eficiencia de la máquina a costa de un control manual más estricto, es decir, dejar un puntero suelto o una llave mal colocada en C++ puede destruir la estabilidad de la aplicación, lo que resalta la importancia de los destructores virtuales para mantener la memoria RAM limpia.

Por último, la resiliencia del sistema quedó probada a través de pruebas negativas, al proteger los flujos de entrada de datos contra errores de tipo y asegurar que la persistencia en los archivos txt sea inmediata, convirtiendo un script frágil en un sistema que sea tolerante a fallos, capaz de resistir a cierres abruptos o a apagones sin llegar a perder un solo registro.

